



كلية العلوم و التقنيات
FACULTÉ DES SCIENCES ET TECHNIQUES



Rapport de projet

Naviguer un Labyrinthe

UNIVERSITE DE NOUAKCHOTT | FACULTE DES SCIENCES ET TECHNIQUES

M1 | IA-MLDS | 01 mai 2025

Mohamadou Ba – C20509 | Saviya Mohamed - C14417 | Cheikh Tijani - C30875

🎯 Objectif du projet

L'objectif principal de ce projet est d'implémenter deux algorithmes de recherche, **BFS** (Breadth-First Search) et **DFS** (Depth-First Search), pour résoudre un labyrinthe représenté sous forme de grille, puis de **visualiser** et **comparer** leurs performances.

🧠 Méthodologie adoptée

1. Représentation du labyrinthe

Le labyrinthe est codé comme une matrice 2D dans un fichier séparé (`my_maze.py`) où :

- **0** représente une case vide (chemin possible),
- **1** représente un mur (obstacle),
- **S** indique la position de départ,
- **G** indique la position d'arrivée.

Un objet `Maze` a été défini pour gérer cette représentation et fournir les **voisins valides** (cas accessibles autour d'une position donnée).

2. Implémentation des algorithmes

- **BFS** : utilise un fichier pour explorer tous les voisins d'un nœud avant d'aller plus loin. Il est **garanti de trouver le plus court chemin** en nombre d'étapes si le coût est uniforme.
- **DFS** : utilise la récursivité pour explorer un chemin jusqu'au bout avant de revenir en arrière. Il est **plus rapide dans certains cas**, mais ne garantit pas d'optimalité.

Les deux algorithmes retournent à la fois :

- La **liste des positions** formant le chemin trouvé,
- L'**ensemble des positions visitées** pour la visualisation.

3. Visualisation

Un module dédié (`visualizer.py`) affiche le labyrinthe avec :

- Les murs en **noir**
- Le départ en **bleu**
- L'arrivée en **orange/gold**.
- Les cas explorés en **gris clair**,
- Le chemin final trouvé en **vert claire**.

Cela permet de visualiser concrètement le fonctionnement de chaque algorithme.

4. Comparaison

À chaque exécution, sur affiche :

- Le **nombre de cas visités**,
- La **longueur du chemin trouvée**.

Cela permet une comparaison simple de **l'efficacité** (nombre d'explorations) et de **l'optimalité** du chemin.

⚠ Problèmes rencontrés et solutions

- **Erreur de type (TypeError)** : Une confusion entre la méthode `maze.neighbors` et l'ensemble `visited` a provoqué une erreur. La solution a été de bien distinguer les **méthodes** et les **valeurs retournées**.
- **Chargement dynamique du fichier labyrinthe** : Pour permettre à l'utilisateur de passer un fichier `.py` contenant la matrice, nous avons utilisé `importlib.util` pour charger dynamiquement les données.
- **Visualisation bloquée** : `matplotlib` interrompt parfois l'exécution en attente de la fermeture de la fenêtre. Cela a été résolu en isolant l'affichage dans une fonction dédiée (`show_path`) qui s'exécute uniquement si l'option `--visual` est activée.

☑ Résultats obtenus

- BFS trouve toujours le chemin le plus court, mais explore souvent plus de cases.
 - DFS explore souvent moins de cas, mais le chemin trouvé est parfois plus long ou sous-optimal.
 - La visualisation permet de bien voir la différence entre les deux stratégies de recherche.
-

📌 Conclusion

Ce projet nous a permis de mettre en œuvre deux algorithmes classiques de recherche, de comprendre leur fonctionnement pratique à travers la visualisation, et de comparer leurs performances sur un même labyrinthe. Il constitue une bonne base pour étendre à d'autres algorithmes comme A* ou Dijkstra.
